

MICROSERVIÇOS

1. INTRODUÇÃO

A arquitetura de software evoluiu de modelos centralizados para sistemas distribuídos. O modelo tradicional, conhecido como Monólito, agrupa todas as funcionalidades de uma aplicação (autenticação, banco de dados, interface e lógica de negócio) em uma única unidade de implantação. Embora simples de iniciar, o monólito apresenta desafios de escalabilidade, pois exige que todo o sistema seja replicado mesmo que apenas uma função específica esteja sob alta carga.

Em contrapartida, a arquitetura de Microserviços decompõe a aplicação em serviços independentes que se comunicam via protocolos leves, como HTTP. Os principais pilares desta abordagem são:

- 1. Escalabilidade Seletiva:** É possível alocar mais recursos apenas para o serviço que demanda maior processamento.
- 2. Desacoplamento:** Falhas em um serviço não derrubam obrigatoriamente todo o ecossistema.
- 3. Banco de Dados por Serviço:** Cada microserviço possui sua própria base de dados, garantindo o isolamento e a integridade dos dados, evitando que alterações em um esquema afetem serviços não relacionados.

2. STACK TECNOLÓGICA

Para este projeto, utilizaremos ferramentas modernas que permitem o desenvolvimento completo sem custos de licenciamento ou infraestrutura inicial:

- 1. Backend:** Node.js com o framework Express para a criação das APIs.
- 2. Banco de Dados:** MongoDB Atlas, um serviço de banco de dados NoSQL na nuvem.
- 3. Frontend:** React.js utilizando Vite para o build e Tailwind CSS para a estilização.
- 4. Hospedagem:** Render para os serviços de backend e Vercel para a interface frontend.

3. REQUISITOS DO SISTEMA

O sistema consiste em um gerenciador de notas colaborativas com os seguintes requisitos:

Requisitos Funcionais:

- O usuário deve ser capaz de criar uma conta e realizar login (Auth Service).
- O usuário deve ser capaz de criar, listar e excluir notas (Note Service).

Requisitos Não-Funcionais:

- A comunicação entre frontend e backend deve ser protegida por tokens JWT (JSON Web Token).
- O sistema deve estar totalmente funcional na nuvem (Deploy).

4. MÓDULO 1: INFRAESTRUTURA

Infraestrutura de Dados

A configuração do MongoDB Atlas é o primeiro passo para garantir a persistência dos dados.

- 1. Criação da Conta:** Acesse o site do MongoDB Atlas e crie uma conta gratuita.
- 2. Criação do Cluster:** Selecione a opção de deploy gratuito (Shared Cluster) e escolha um provedor (AWS/Google Cloud) e região próxima.
- 3. Database Access:** No menu lateral, clique em Database Access. Adicione um novo usuário com nome e senha. Guarde estas credenciais.
- 4. Network Access:** Clique em Network Access e selecione Add IP Address. Escolha Allow Access From Anywhere (0.0.0.0/0) para permitir a conexão a partir dos servidores de deploy.
- 5. Connection String:** No painel principal (Database), clique em Connect, selecione Drivers e copie a URL de conexão. Substitua a parte da senha pela senha criada no passo 3.

Stack Tecnológico

Interconexão das tecnologias no Sistema de Microserviços

Tecnologia	Papel no Sistema	Analogia Didática
MongoDB Atlas	Banco de Dados (Nuvem)	Responsável pelo armazenamento permanente dos dados.
Node.js + Express	Backend (API)	Recebe as requisições e fornece os dados.
React + Vite	Frontend (Interface)	Responsável pela Interface responsiva com o usuário.
JWT (Token)	Segurança / Autenticação	Responsável pelo Controle de acesso.
Render	Hospedagem Backend	Infraestrutura de servidor onde o backend irá “rodar”.
Vercel	Hospedagem Frontend	Infraestrutura de servidor onde o frontend irá “rodar”.

O ecossistema distribuído, onde cada peça possui autonomia técnica e funcional. A interconexão ocorre através de requisições HTTP, utilizando o formato JSON para o tráfego de dados.

Interconexão das Tecnologias

O fluxo de dados segue uma hierarquia lógica: o Frontend (React) é o ponto de entrada do usuário. Quando uma ação é realizada (como um login), o React envia um pedido para o Auth Service (Backend). Se as credenciais estiverem corretas, o Backend consulta o MongoDB Atlas e devolve um JWT (Token).

A partir daí, o Frontend guarda esse "token" e envia ao “Note Service” sempre que precisar de dados. O “Note Service”, por sua vez, valida o crachá e busca as notas específicas daquele usuário no banco de dados. Todo esse ecossistema é mantido vivo na nuvem pelo Render e pela Vercel, que garantem que os endereços (URLs) de cada serviço sejam acessíveis via internet.

INICIALIZAÇÃO DO PROJETO

Para iniciar o desenvolvimento, organize a estrutura de pastas localmente. Cada serviço deve residir em seu próprio diretório para manter o isolamento da arquitetura.

```
mkdir ufn-microservicos
cd ufn-microservicos

mkdir auth-service note-service frontend
```

Em cada pasta de backend (**auth-service** e **note-service**), inicialize o **Node.js** para criar o arquivo **package.json**:

```
npm init -y
```

INSTALAÇÃO DE DEPENDÊNCIAS

A instalação correta das bibliotecas é fundamental para o funcionamento dos serviços. Execute os comandos abaixo dentro de cada respectiva pasta:

Dependências do Auth Service:

```
cd auth-service
npm install express jsonwebtoken cors
```

Dependências do Note Service:

```
cd ../note-service
npm install express mongoose cors jsonwebtoken
```

Dependências do Frontend:

```
cd ../frontend
npm install axios
```

5. MÓDULO 2: BACKEND (MICROSERVIÇOS)

5.1. AUTH SERVICE (index.js)

Este serviço é responsável por gerenciar a identidade dos usuários.

```
const express = require('express');
const jwt = require('jsonwebtoken');
const cors = require('cors');
const app = express();
```

```
app.use(express.json());
app.use(cors());
```

```
const USERS = []; // Em produção, utilize o MongoDB
const JWT_SECRET = process.env.JWT_SECRET || 'chave_secreta_ufn';
```

```
// Rota de Registro
app.post('/register', (req, res) => {
  const { email, password } = req.body;
  USERS.push({ email, password });
  res.status(201).send({ message: 'Usuario criado com sucesso' });
});

// Rota de Login
app.post('/login', (req, res) => {
  const { email, password } = req.body;
  const user = USERS.find(u => u.email === email && u.password === password);

  if (user) {
    const token = jwt.sign({ email }, JWT_SECRET, { expiresIn: '1h' });
    return res.json({ token });
  }
  res.status(401).send({ message: 'Credenciais invalidas' });
});

app.listen(3001, () => console.log('Auth Service rodando na porta 3001'));
```

5.2. NOTE SERVICE (index.js)

Este serviço gerencia as notas e valida o acesso via token.

```
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const jwt = require('jsonwebtoken');
const app = express();

app.use(express.json());
app.use(cors());

const JWT_SECRET = process.env.JWT_SECRET || 'chave_secreta_ufrn';
const MONGO_URI = process.env.MONGO_URI;

mongoose.connect(MONGO_URI)
  .then(() => console.log('Conectado ao MongoDB'))
  .catch(err => console.error('Erro ao conectar ao MongoDB', err));

const NoteSchema = new mongoose.Schema({
  content: String,
  userEmail: String
});

const Note = mongoose.model('Note', NoteSchema);

// Middleware de Autenticacao
const authenticate = (req, res, next) => {
  const token = req.headers['authorization'];
  if (!token) return res.status(403).send('Token obrigatorio');
```

```

try {
  const decoded = jwt.verify(token, JWT_SECRET);
  req.user = decoded;
  next();
} catch (err) {
  res.status(401).send('Token invalido');
}
};

// Rotas de Notas
app.get('/notes', authenticate, async (req, res) => {
  const notes = await Note.find({ userEmail: req.user.email });
  res.json(notes);
});

app.post('/notes', authenticate, async (req, res) => {
  const newNote = new Note({
    content: req.body.content,
    userEmail: req.user.email
  });
  await newNote.save();
  res.status(201).json(newNote);
});

app.listen(3002, () => console.log('Note Service rodando na porta 3002'));

```

6. MÓDULO 3: FRONTEND

6.1. APP.JSX (REACT)

Interface para interação do usuário com os microserviços.

```

import React, { useState, useEffect } from 'react';
import axios from 'axios';

function App() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [token, setToken] = useState(localStorage.getItem('token'));
  const [notes, setNotes] = useState([]);
  const [newNote, setNewNote] = useState('');

  const API_AUTH = 'URL_DO_AUTH_SERVICE';
  const API_NOTES = 'URL_DO_NOTE_SERVICE';

  const handleLogin = async () => {
    try {
      const res = await axios.post(`${API_AUTH}/login`, { email, password });
      setToken(res.data.token);
      localStorage.setItem('token', res.data.token);
    } catch (err) {

```

```

    alert('Erro no login');
  }
};

const fetchNotes = async () => {
  const res = await axios.get(`${API_NOTES}/notes`, {
    headers: { Authorization: token }
  });
  setNotes(res.data);
};

const addNote = async () => {
  await axios.post(`${API_NOTES}/notes`, { content: newNote }, {
    headers: { Authorization: token }
  });
  setNewNote("");
  fetchNotes();
};

useEffect(() => {
  if (token) fetchNotes();
}, [token]);

if (!token) {
  return (
    <div style={{ padding: '20px' }}>
      <h1>Login - Sistema UFN</h1>
      <input type="email" placeholder="Email" onChange={e => setEmail(e.target.value)} />
      <input type="password" placeholder="Senha" onChange={e => setPassword(e.target.value)} />
      <button onClick={handleLogin}>Entrar</button>
    </div>
  );
}

return (
  <div style={{ padding: '20px' }}>
    <h1>Minhas Notas</h1>
    <input value={newNote} onChange={e => setNewNote(e.target.value)} placeholder="Nova nota..." />
    <button onClick={addNote}>Salvar</button>
    <ul>
      {notes.map(note => (
        <li key={note._id}>{note.content}</li>
      ))}
    </ul>
    <button onClick={() => { localStorage.clear(); setToken(null); }}>Sair</button>
  </div>
);
}

export default App;

```

7. MÓDULO 4: DEPLOY E INTEGRAÇÃO

TESTES LOCAIS

Antes de realizar o deploy, valide a aplicação em ambiente de desenvolvimento local seguindo estes passos:

1. Configuração de Variáveis: Certifique-se de que as variáveis de ambiente como **MONGO_URI** e **JWT_SECRET** estejam configuradas ou passadas via terminal.

2. Execução dos Serviços: Abra três terminais diferentes e inicie cada parte do sistema:

Terminal 1 (Auth Service)

```
cd auth-service  
node index.js
```

Terminal 2 (Note Service)

```
cd note-service  
node index.js
```

Terminal 3 (Frontend)

```
cd frontend  
npm run dev
```

3. Validação de Fluxo: Acesse o **localhost** indicado pelo Vite, registre um novo usuário, realize o login e tente criar uma nota para verificar a persistência no **MongoDB Atlas**.

7.1. PREPARAÇÃO NO GITHUB

1. Crie três repositórios no GitHub: um para o Auth Service, um para o Note Service e um para o Frontend.
2. Faça o push dos códigos desenvolvidos para seus respectivos repositórios.

7.2. DEPLOY DO BACKEND NO RENDER

Repita este processo para o Auth Service e para o Note Service:

1. Acesse o painel do Render e clique no botão azul + New.
2. Selecione a opção Web Service.
3. Conecte sua conta do GitHub e selecione o repositório do serviço.
4. No campo Runtime, selecione Node.

5. No campo Build Command, digite: npm install.
6. No campo Start Command, digite: node index.js.
7. Clique em Advanced e depois em Add Environment Variable.
8. Adicione as variáveis: JWT_SECRET (uma senha qualquer) e MONGO_URI (a string do Atlas).
9. Clique em Create Web Service. Ao final, o Render fornecerá uma URL (ex: <https://auth-service.onrender.com>).

7.3. DEPLOY DO FRONTEND NA VERCEL

1. Acesse a Vercel e clique em Add New e depois em Project.
2. Importe o repositório do Frontend.
3. Em Framework Preset, a Vercel detectará automaticamente o Vite.
4. Em Environment Variables, adicione as URLs dos serviços criados no Render (ex: VITE_API_AUTH e VITE_API_NOTES).
5. Clique em Deploy.

Atenção

Atenção ao Cold Start: Como estamos utilizando o plano gratuito do Render, os serviços entram em modo de suspensão após 15 minutos de inatividade. Ao acessar o sistema pela primeira vez no dia, a primeira requisição pode demorar até 30 segundos para ser processada enquanto o servidor "acorda".

Explicação dos Códigos

Para facilitar a compreensão, podemos dividir a lógica em três pilares:

1. Auth Service: O Porteiro do Sistema

O código deste serviço foca em identificar o usuário.

- Rota de Login: Imagine que o usuário apresenta seu RG e senha. O código verifica se esses dados batem com o que está no banco.
- Geração do JWT: Se os dados estiverem corretos, o servidor assina um "ticket digital" (Token). Esse ticket contém o ID do usuário de forma criptografada, para que o sistema saiba quem ele é sem precisar perguntar a senha novamente.

2. Note Service: O Cofre de Anotações

Este serviço é mais rigoroso. Ele possui uma função chamada Middleware de Autenticação.

- O Middleware: Antes de qualquer ação (listar ou criar notas), essa função intercepta o pedido e pergunta: "Cadê o seu ticket?". Se o Token for válido, a requisição prossegue; caso contrário, o usuário é barrado com um erro 401 (Não autorizado).
- Filtro de Dados: Ao buscar as notas, o código não traz todas as notas do banco, mas apenas aquelas que possuem o userEmail ou userId correspondente ao dono do Token.

3. Frontend (React): Experiência do Usuário

O código no React é responsável por gerenciar o estado da aplicação.

- **localStorage:** Quando o usuário recebe o Token, o React o salva no navegador. Isso permite que, mesmo que a página seja atualizada, o usuário continue "logado".
- **Axios e Headers:** Nas chamadas para buscar notas, o React configura um cabeçalho (Header) de autorização. É como se ele estivesse grampeando o ticket digital em cada envelope de pedido enviado ao servidor.
- **Renderização Condicional:** O código verifica se existe um Token. Se não existir, ele mostra a tela de Login; se existir, ele mostra o painel de Notas.